PROJECT-BASED LEARNING

PHASE-II EVALUATION

Team ID: DSCPP-III-2026-T013

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026–27

Team & Mentor Details

Team Information

Team ID: DSCPP-III-2026-T013
Semester: 3rd
Section: BTECH
Domain: *CSE*

Team Members

1. *Vanshika Aggarwal – 2510011410*
2. *Suvidha Setia – 2510370306*
3. *Parishi Aggarwal – 2510011423*
4. *Khushi Taneja – 2510379518*

Mentor

Ms. Pratibha Waldia

Interactions completed:8

Phase-I Recap & Feedback Incorporated

Phase-I Status

We identified the challenges students and visitors face while finding buildings on campus and proposed a Smart Campus Navigation System using graphs and Dijkstra's algorithm. We planned the system architecture, identified campus locations, and decided how to calculate the shortest route between two buildings.

Feedback Received

- *Improve accuracy of campus spots and paths.*
- *Ensure routes follow real walkable walkways.*
- *Make the system easy for users.*

Changes Incorporated

- *Mapped spots and surveyed real walking paths.*
- *Built a weighted graph with Dijkstra in C++.*
- *Started web UI for navigation.*

Problem Statement & Finalized Objectives

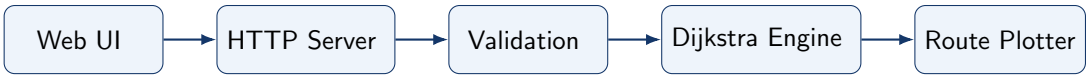
Problem Statement

New students, visitors, and new faculty often struggle to navigate between buildings at Graphic Era because commercial maps only estimate straight-line distances and do not reflect internal pedestrian walkways. Our goal is to solve this by providing a straightforward a web app that requires only a starting building and a destination, eliminating confusing indoor details and giving clear, reliable walking directions.

Finalized Objectives

1. *Map Walking Routes: Map nodes and walking paths.*
2. *Compute Shortest Paths: Run Dijkstra's algorithm in C++ for optimal routes.*
3. *Connect Frontend & Backend: Use httplib.h to serve JSON route data via HTTP.*
4. *Build Simple Web UI: Take user inputs and visually display the path.*

Updated System Architecture / Workflow



Major Components

- *Frontend UI: User inputs and route display.*
- *HTTP Server: httplib.h handling JSON requests.*
- *Graph Engine: C++ backend running Dijkstra.*

Changes Since Phase-I

- *Web Setup: Moved from CLI to client-server model.*
- *REST API: Added HTTP layer for data flow.*
- *Visual Output: Added on-screen graph rendering.*

Module Design & Functional Requirements

Major Modules

1. *Web UI:index.html*
2. *Client Script: js/*
3. *HTTP Server: ApiServer.h, ApiServer.cpp, main.cpp*
4. *Graph Engine: Graph.h, Graph.cpp, Node.h,Edge.h*

Functional Requirements

- *Accept start and destination names.*
- *Validate location names*
- *Compute the shortest path using real walking distances.*
- *Return and highlight the path sequentially on the campus map.*

Module Dependency

The UI triggers client scripts to send user inputs to the HTTP server; the server validates names and queries the Graph Engine, returning the shortest path to the UI for visual map rendering.

Technology Stack & Development Methodology

Technology Stack

- Programming: *C++17, JavaScript, HTML, CSS*
- Database: *In-memory graph (std::map, std::vector)*
- Framework: *httplib.h, Vite*
- Development Tool: *GCC/MinGW, VS Code, Node.js*
- Version Control: *Git, GitHub*

Development Methodology

1. Surveyed nodes & walkable paths
2. Modeled Location & Graph structures
3. Built Dijkstra backend & UI scripts
4. Linked C++ & JavaScript via REST API
5. Verified actual walking route outputs

Implementation of PBL Course Concepts

Course	Concepts Actually Implemented	Project Module
Data Structures in C	*Adjacency List, Dijkstra, Priority Queue*	*Graph.cpp, Graph.h*
OOPs with C++	*Classes, Encapsulation, STL (vector, map, pair)*	*Node.h,Edge.h*

Development & Implementation Progress

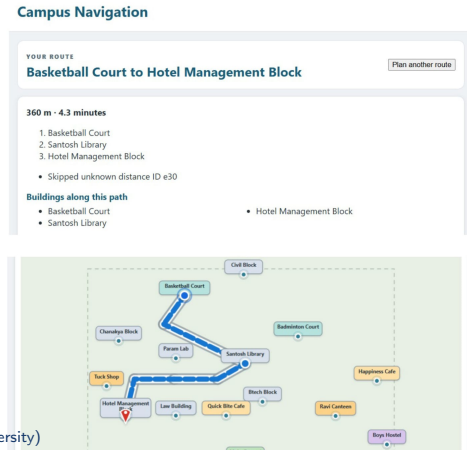
Module	Status	Work Completed
Graph Model	Completed	*Mapped nodes, edges with actual walking distances.*
Dijkstra Engine	Completed	*Built C++ priority-queue shortest-path algorithm.*
HTTP Server & API	In Progress	*Added httplib.h server to parse JSON and route requests.*
Field Testing	Pending	*Validate route accuracy using actual campus walks.*

Overall Project Completion: *70%*

Working Prototype / Demonstration

Implemented Features

- *Location Selection: Start and destination drop-down building inputs.*
- *Input Validation: Real-time error checks for invalid building names.*
- *Shortest Path Calculation: C++ Dijkstra solver returning shortest routes.*
- *Visual Route Highlighting: Dynamic path drawing across campus nodes*



Testing, Results & Validation

Test / Module	Expected Result	Observed Result	Status
Shortest Path	*Find the shortest walking route between two campus locations.*	*Shortest route displayed using Dijkstra's algorithm.*	Pass
Invalid Location	*Show an error for an invalid starting or destination location.*	*Invalid location is rejected with an error message.*	Pass
Same Start & Destination	*Handle the selection of the same location correctly.*	*Display a message indicating that the user is already at the destination.*	*Pass*

Results / Validation

The system maps multiple campus locations and walking paths using a weighted graph. Dijkstra's algorithm calculates the shortest walking route, while input validation and route visualization are tested. Performance is evaluated using route distance and computation time.

GitHub Repository & Development Activity

Repository Information

- `https://github.com/suvidhasetia14-creator/DSCPP-III-2026-T013`
- README.md: *Updated with features and technologies used.*
- Contributors: *All four team members included.*
- Repository Status: *Latest implementation uploaded.*
- Branches: *Main branch and development branches.*

Development Activity

- *Regular commits maintained throughout the development process.*
- *Meaningful commit messages used to describe changes.*
- *Individual contributions tracked through GitHub commit history.*
- *Latest implementation pushed to the repository.*
- *Project documentation updated regularly.*

Individual Contribution

Member	Role	Technical Contribution	GitHub Commits
Vanshika Aggarwal	*Backend Developer*	*Developed the C++ graph structure and implemented Dijkstra's shortest-path algorithm.*	*16*
Suvidha Setia	*Frontend Developer*	*Designed and developed the web interface using HTML, CSS and JavaScript for selecting locations and displaying navigation routes.*	*15*
Parishi Aggarwal	*Graph & Campus Mapping*	*Mapped campus locations and walking paths and helped build the weighted graph for route calculation.*	*16*
Khushi Taneja	*Software Testing & Documentation Lead*	*Tested navigation functionality and route outputs, identified errors, and prepared project documentation.*	*15*

Challenges, Remaining Work & Phase-III Roadmap

Challenges Faced

- *Graph Modeling: Mapping irregular campus paths into 19 nodes.*
- *C++ Web Bridge: Connecting C++ memory to HTTP REST requests.*
- *Input Edge Cases: Handling name typos and invalid locations.*

Solutions / Learning

- *Adjacency Lists & STL: Leveraged unordered map for fast lookups.*

Phase-III Roadmap

1. *Interactive UI: Finalize animated canvas path drawing.*
2. *Dynamic Weights: Add stair-free, accessible path options.*
3. *Field Testing: Validate routes via real campus walks.*
4. *Local Deployment: Package executable for single-click launch.*
5. *Final Report: Document system benchmarks and project metrics.*

Expected Outcomes & References

Expected Final Outcomes

- *Live System: Working web tool for campus navigation.*
- *Fast Routing: Instant Dijkstra shortest-path calculations.*
- *Campus Use: Practical daily navigation for visitors and students.*
- *Easy Expansion: Simple addition of new paths and nodes.*
- *Modular Architecture: Decoupled C++ core and web frontend.*

References

1. *E. W. Dijkstra, "A Note on Two Problems in Connexion with Graphs" (1959).*
2. *T. H. Cormen et al., Introduction to Algorithms (CLRS).*
3. *cpp-httplib Documentation (GitHub).*
4. *ISO/IEC 14882:2017 C++17 Standard.*
5. *https://github.com/suvidhasetia14-creator/DSCPP-III-2026-T013*